Département Informatique
Promotion 2022-2025

Librairie Graphique C++/Java

Rapport technique individuel de

Dorian BUCCHIOTTY

Projet effectué au département informatique de l'IUT de Metz

Introduction

Le projet consiste à la création d'une librairie graphique pour le langage C++ le besoin se trouve à l’inexistence de libraire graphique officiel à ce langage, le but est de combler ce manque en créer cette librairie. Ce rapport individuel vise à présenter l'état actuel du projet, de voir d’un point de vue individuel ce projet, les choix technologiques et les principales étapes de son en cours développement.

Les parties sur lequel j’ai le plus travaillé sont :

La classe Fenetre en C++
La librairie AWT
Le mapping to viewport

Analyse, Conception

Pour ce projet nous avons analysé les besoins qui sont clairs, le problème est que le C++ n'a aucune librairie officiel, donc nous devons élaborer une librairie Graphique pour le C++ qui permettra de dessiner en C++, la librairie graphique que nous devons utiliser est celle de java est plus précisément AWT.

Pour la conception nous avons établies les principales tâches qui sont :

- La communication entre les deux programmes (C++/Java)
- La création d'un objet C++ pour utiliser une fenêtre
- La création d'un objet qui traitera les requêtés pour chaque fenêtre créer coté java

Diagramme de cas d'utilisation :

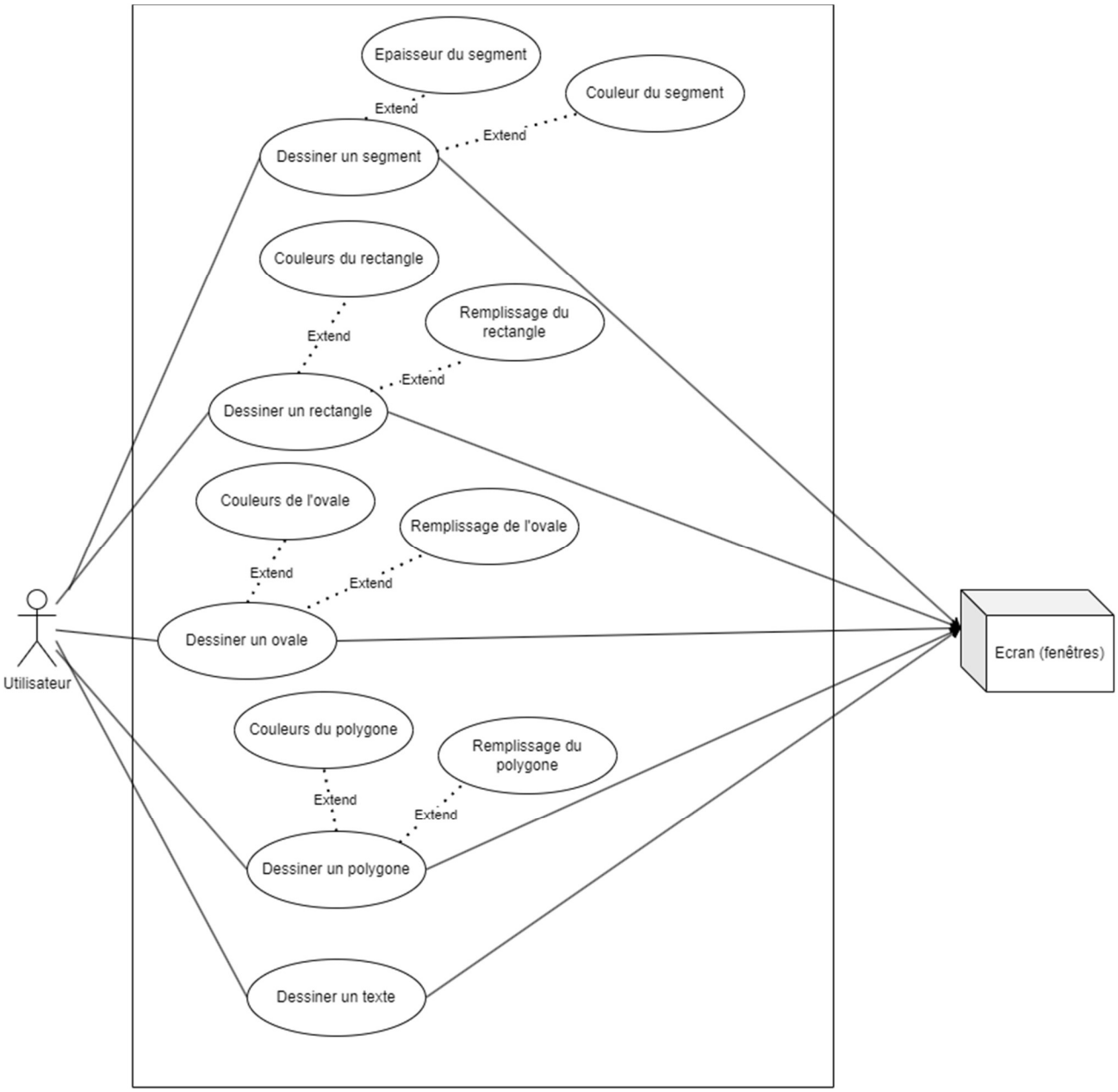


Diagramme gantt project :

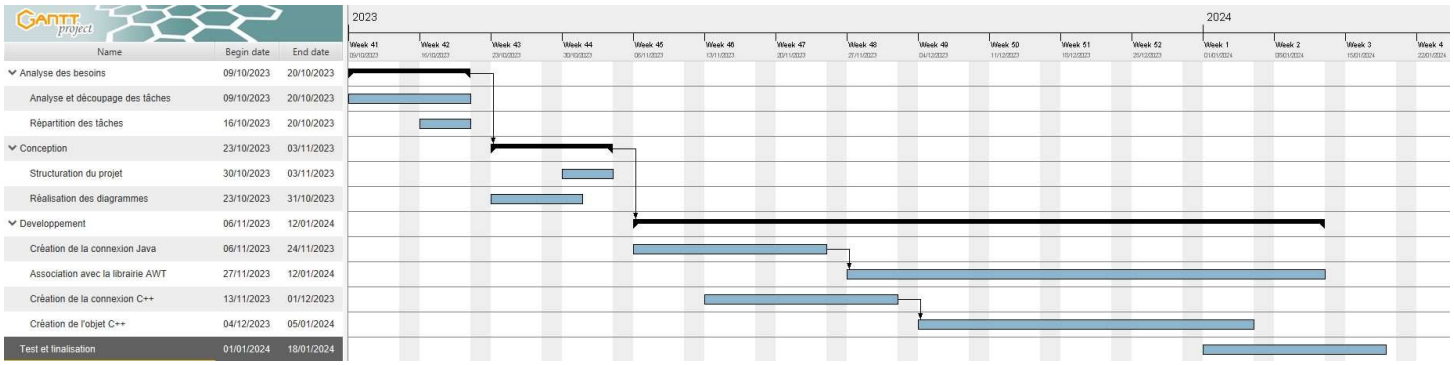


Diagramme de PERT :

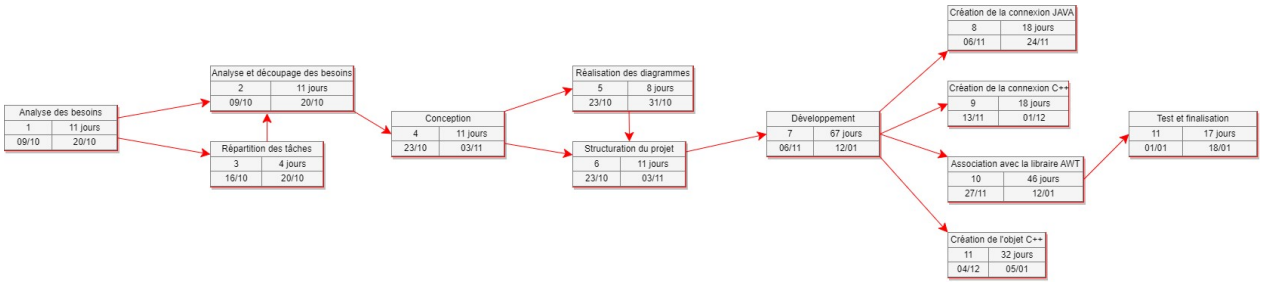


Diagramme de classe :

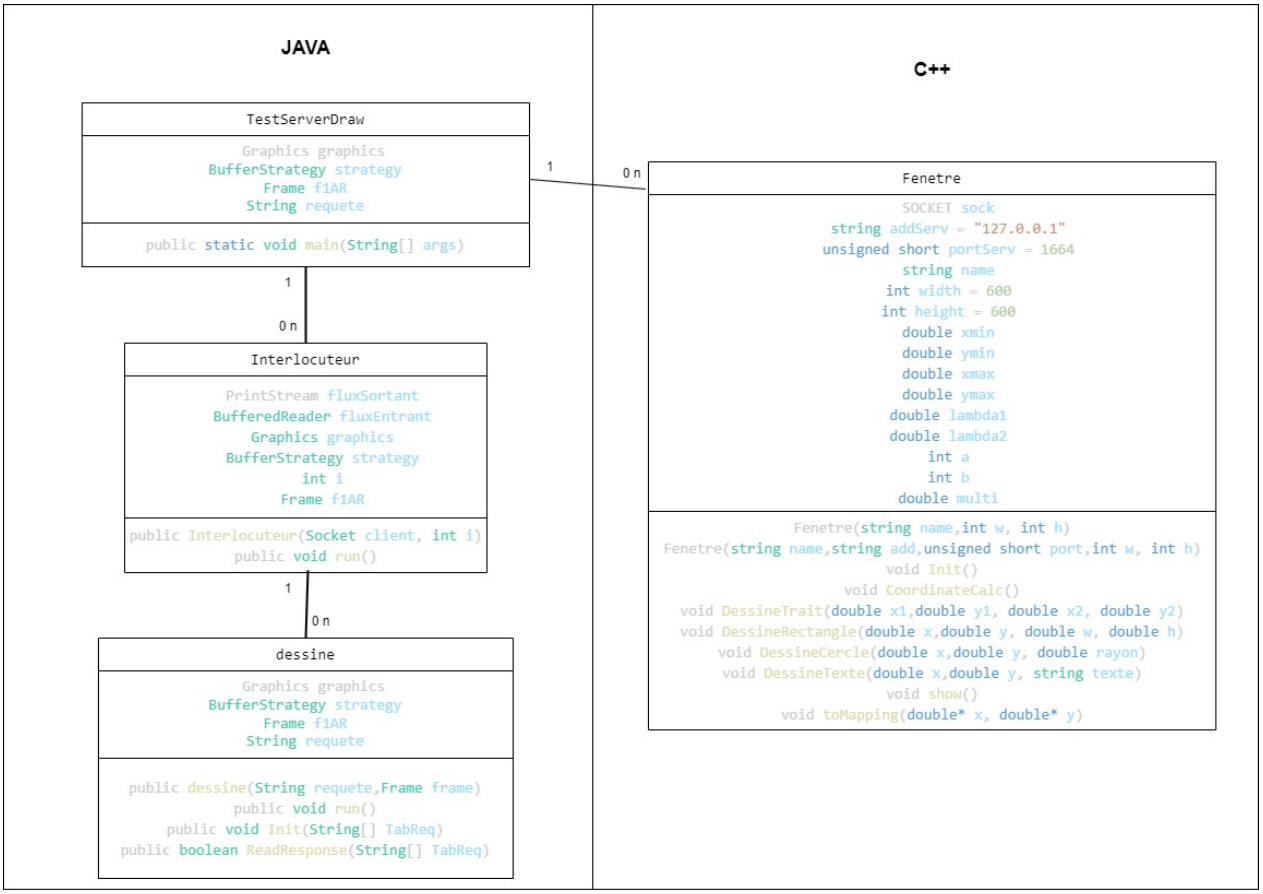
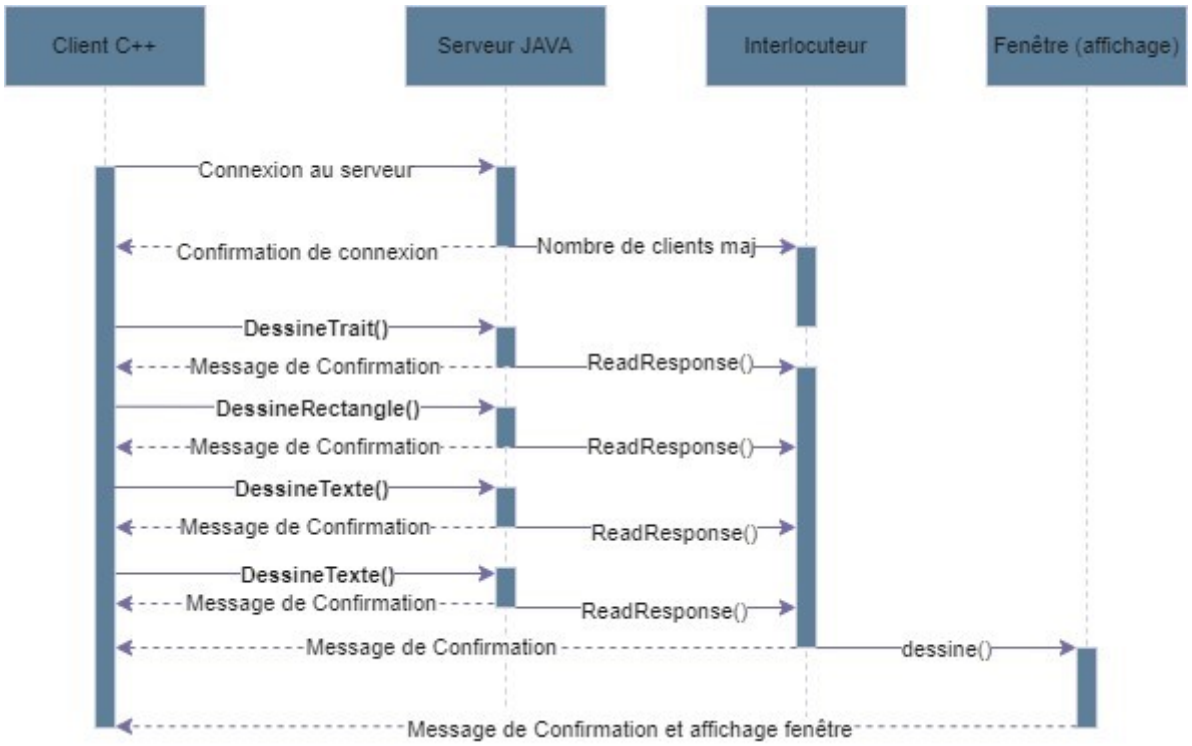


Diagramme de séquence :



Développement, Réalisation

Communication Client C++ à Server Java

Pour la communication nous avons utilisé le système de socket. Pour le client C++ nous avons utilisé la librairie « winsock.h » pour Windows et pour Linux les librairies « sys/socket.h » et « netinet/in.h » et pour la partie Server Java on appelle la librairie « java.net »

Nous avons suivi des instructions pour pouvoir utiliser les sockets en C++ avec des exemples pour comprendre comment les utilisés (voir annexe 2.0), j'ai eu quelque problème telle que « char* » sont assez complexe à utiliser, pour y remédier j'ai utilisé un « String » et dès que nous avons besoin d'un « char* » j'utiliser la méthode de la classe String .c_str() qui retourne un « char* » qui nous convient parfaitement.

Exemple :

```
SOCKET Connexion(string addr,unsigned short* port)
```
// entête de la fonction

```
sockaddr.sin_addr.s_addr = inet_addr(addr.c_str());
```
// utilisation de addr

Un deuxième problème à était l'utilisation de la librairie Erreur.h (et Erreur.cpp), La classe permet de throw une erreur proprement, nous avions fait quelque erreur dans le code, puis à chaque utilisation de g++ j'oubliais de compiler Erreur.cpp ce qui provoqua une erreur :

```
C:\Users\bucch\AppData\Local\Temp\cc7rvyfx.o:fenetre.class.cpp:(.text+0x14e
): undefined reference to `Erreur::Erreur(char const*)'
collect2.exe: error: ld returned 1 exit status
```

Coté java la gestion des sockets ce fait plus facilement hormis que java est la partie serveur donc nous avons eu besoin d'utiliser les threads pour pouvoir géré toutes les demandes des clients. Pour ce ferais nous avons une créer une class Interlocuteur (voir annexe 3.1) qui sera instancier est lancé sur un thread pour chaque client.

```
Interlocuteur interlocuteur = new Interlocuteur(client,i);
interlocuteur.start();
```

Java Server

La partie java consiste à faire un server Java qui attend des requêtes qui permettront de faire apparaitre une fenêtre sur laquelle on va dessiner
Pour ça nous avons utilisé la librairie Awt de Java.
La librairie permet de créer une fenêtre, elle permet aussi de créer des rendus sur

lesquelles nous pourrons dessiner. Elle a un tas de fonction qui dessine, certaine de ses fonctions permette de dessiner des segments, rectangle, ovale, arc, texte.
Chaque fonction possède ses propres paramètres mais ils ont tous en commun besoin des coordonnées en x et y.
On pourra pour chaque cas choisir la couleur par rapport à sa forme,
Et d'autre choix comme l'épaisseur pour un segment ou si une forme fermée doit être remplis.

C++ Client

La partie C++ consiste à faire un client C++ qui va envoyer des requêtes au server, l'utilité est que le C++ à une vitesse de calcule largement plus rapide que le java

Comme nous voulons une librairie l'utilisation des méthodes doivent être simple d'utilisation et intuitive (voir annexe 2.1), pour le faire nous avons structuré de telle sorte que l'utilisateur n'utilise qu'un objet, cette objet est une instanciation de la class Fenetre (voir annexe 2.3)
Pour commencer nous avons initialisation l'utilisateur commence par entrée les informations de la fenêtre voulu. (nom de la fenêtre, largeur, hauteur)

```
Fenetre(string name,int w, int h)
```

Une fois les informations enregistrées nous nous connectons au serveur java, si aucune erreur ne s'est produite nous envoyons les caractéristiques de la fenêtre voulu au serveur java, ensuite java créer la fenêtre. Pendant ce temps nous calculons les variables qui nous serons utile pour le « mapping coordinate to viewport » (voir annexe 2.2 méthode=> Init()).

Ensuite pour dessiner sur la fenêtre java nous devons simple utiliser les méthodes de l'objet instancier de la classe Fenetre.

Exemple :

```
Fenetre frame("NomPage", 600, 600);
frame.DessineTrait(0, 0, 0.5, 0.5);
frame.DessineTrait(0.6, 0.6, 1, 1);
frame.DessineTrait(1.1, 1.1, 1.5, 1.5);
frame.DessineTrait(1.6, 1.6, 2, 2);
```

Mapping to viewport

Le mapping to viewport à était fait dans un premier temps du coté de C++ mais j'ai constaté quelques defaults, donc j'ai décidé de le mettre du coté de java pour résoudre c'est default.

Le principe du mapping to viewport c'est d'enregistre des dimensions dans n'importe quel unité de mesure km, m, cm, miles, etc… Et de pouvoir l'afficher sur l'écran en pixel donc nous devons transformés les unités en pixel pour pouvoir l'afficher.

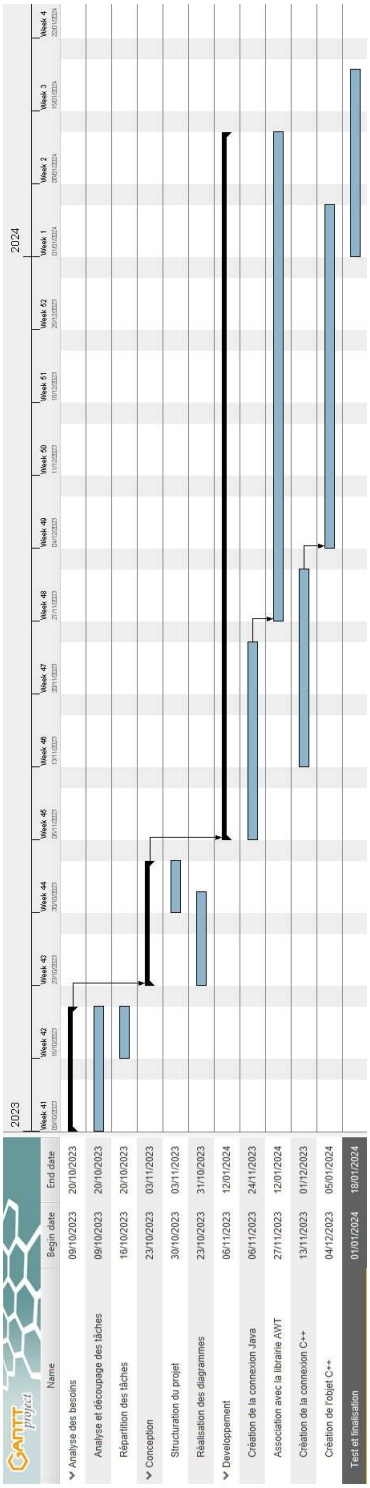
Exemple : cm → pixel
Il faut aucune déformation en restant proportionnelles pour l'afficher sur n'importe quel fenêtre peut importe sa taille ou sa résolution d'écran.

Conclusion

Nous devons continuer pour pouvoir produire une librairie de qualité qui conviendra la demande, il faudra corriger les problèmes et améliorer les solutions trouvé.

Annexe

c.f 1.1



c.f 2.0

```
SOCKET Connexion(string addr,unsigned short* port)
{
    try
    {
        //libraire winsock
        int r;
        WSADATA wsaData;
        r = WSAStartup(MAKEWORD(2, 0), &wsaData);

        if(r){
            throw Erreur("Echec de l'initialisation");
        }
        //création d'un socket
        SOCKET sock;

        int familleAdresses = AF_INET; //IPv4
        int typeSocket = SOCK_STREAM;
        int protocole = IPPROTO_TCP;

        sock = socket( familleAdresses, typeSocket, protocole );

        if(sock == INVALID_SOCKET) throw Erreur("La creation du socket a echouee : code d'erreur = " + WSAGetLastError());
        // cout << "La creation du socket a reussie\n";

        //création d'un représent local du serveur distant
        SOCKADDR_IN sockaddr;

        sockaddr.sin_family = AF_INET;
        sockaddr.sin_addr.s_addr = inet_addr(addr.c_str()); //inet_addr() convertit l'ASCII en entier
        sockaddr.sin_port = htons(*port); //htons() s'assure que le port est bien inscrit dans le format réseau (little-endian ou big-endian)

        //connexion au serveur
        r = connect(sock, (SOCKADDR * ) &sockaddr, sizeof(sockaddr));
        if(r == SOCKET_ERROR){
            throw Erreur("Echec de la connexion au serveur de majuscule");
        }
        // cout << "Connexion au serveur de majuscule reussie\n";

        return sock;
    }
    catch(const Erreur e)
    {
        cerr << e.message << '\n';
    }
    return SOCKET_ERROR;
}
```

c.f 2.1

```
void DessineTrait(double x1,double y1, double x2, double y2){
    this->toMapping(&x1,&y1);
    this->toMapping(&x2,&y2);
    string requete = "droite," + to_string((int)x1) + "," + to_string((int)y1) + "," + to_string((int)x2) + "," + to_string((int)y2)+"\r\n" ;
    send(this->sock, requete.c_str(), requete.length(), 0);
    Sleep(1);
}

void DessineRectangle(double x,double y, double w, double h){
    this->toMapping(&x,&y);

    string requete = "rectangle," + to_string((int)x) + "," + to_string((int)y) + "," + to_string((int)(w*this->multi)) + "," + to_string((int)(h*this->multi))+"\r\n" ;
    send(this->sock, requete.c_str(), requete.length(), 0);
    Sleep(1);
}

void DessineCercle(double x,double y, double rayon){
    this->toMapping(&x,&y);
    string requete = "cercle," + to_string((int)x) + "," + to_string((int)y) + "," + to_string((int)rayon*2) + "," + to_string((int)rayon*2)+"\r\n" ;
    send(this->sock, requete.c_str(), requete.length(), 0);
    Sleep(1);
}

void DessineTexte(double x,double y, string texte){
    this->toMapping(&x,&y);
    string requete = "text," + texte + "," + to_string((int)x) + "," + to_string((int)y)+"\r\n" ;
    send(this->sock, requete.c_str(), requete.length(), 0);
    Sleep(1);
    // if(send(this->sock, requete.c_str(), requete.length(), 0) != SOCKET_ERROR){
    //     cout << requete.c_str();
    // };
}
```

c.f 2.2

```
void CoordinateCalc(){ // calcule les valeur lambda1, lambda2, a et b
    vector<double> point = { this->xmax-this->xmin,this->ymax-this->ymin};
    vector<double> pointPrime = { (double)this->width,(double)-this->height};
    double l1 = abs(pointPrime[0])/abs(point[0]);
    double l2 = abs(pointPrime[1])/abs(point[1]);
    double lambda;
    if(l1 > l2) lambda = l2;
    else lambda = l1;
    short eta1;
    short eta2;
    if(pointPrime[0] > 0 && point[0] > 0) eta1 = 1;
    else eta1 = -1;
    if(pointPrime[1] > 0 && point[1] > 0) eta2 = 1;
    else eta2 = -1;
    this->lambda1 = lambda*eta1;
    this->lambda2 = lambda*eta2;
    vector<double> Cpoint = { (this->xmax+this->xmin)/2,(this->ymax+this->ymin)/2};
    vector<double> CpointPrime = { (double)this->width/2,(double)this->height/2};
    this->a = CpointPrime[0]-(this->lambda1)*Cpoint[0];
    this->b = CpointPrime[1]-(this->lambda2)*Cpoint[1];

    this->multi = abs(this->lambda1); // multiplicateur
}
```

```
void toMapping(double* x, double* y){
    *x = this->lambda1*(*x)+this->a;
    *y = this->lambda2*(*y)+this->b;
}
```

c.f 2.3

```
class Fenetre{
private:
    SOCKET sock;
    string addServ = "127.0.0.1";
    unsigned short portServ = 1664;
    string name;
    int width = 600;
    int height = 600;
    // partie mapping (Tous en brute)
    double xmin = 0;
    double ymin = -2;
    double xmax = 5;
    double ymax = 1;
    double lambda1;
    double lambda2;
    int a;
    int b;
    double multi;


public:
    Fenetre(string name,int w, int h)···

    Fenetre(string name,string add,unsigned short port,int w, int h)···

    void Init(){···

    void CoordinateCalc(){ // calcule les valeur lambda1, lambda2, a et b···

    void DessineTrait(double x1,double y1, double x2, double y2){···

    void DessineRectangle(double x,double y, double w, double h){···

    void DessineCercle(double x,double y, double rayon){···

    void DessineTexte(double x,double y, string texte){···

    void show(){···

    void toMapping(double* x, double* y){···
};
```

c.f 3.0

```java
public class TestServerDraw {
    Run | Debug
    public static void main(String[] args) {

        int portServeur = 1664;
         try {

             ServerSocket server = new ServerSocket(portServeur);
             System.out.println("Server JavaDraw prêt : "+ server);
             int i = 0;
             boolean launch = true;
             while(launch){
                 Socket client = server.accept();
                 ++i;
                 System.out.println("Connection numero "+ i + " réussi");

                 Interlocuteur interlocuteur = new Interlocuteur(client,i);
                 interlocuteur.start();
             }
             server.close();

         }
         catch (Exception e) {
             e.printStackTrace();
         }
    }
}
```

c.f 3.1

```java
public class Interlocuteur extends Thread
{
    PrintStream fluxSortant;
    BufferedReader fluxEntrant;
    Graphics graphics;
    BufferStrategy strategy;
    int i;
    Frame f1AR;

    public Interlocuteur(Socket client, int i) throws IOException
    {
        this.f1AR = new Frame("Page"+i);
        this.f1AR.setSize(400,400);
        this.f1AR.setLocationRelativeTo(null);
        this.f1AR.setIgnoreRepaint(true);
        //parti socket
        //this.fluxSortant = new PrintStream(client.getOutputStream());
        this.fluxEntrant = new BufferedReader(new InputStreamReader(client.getInputStream()));
        this.i = i;
    }

    @Override
    public void run()
    {
        try
        {
            while(!Thread.interrupted())
            {
                String requete = fluxEntrant.readLine();

                System.out.println("Le clientb n°" + i + ", Requete = " + requete);
                dessine th = new dessine(requete, this.f1AR);
                th.start();
            }
            this.graphics.dispose();

        }
        catch(Exception e){}
    }
}
```

c.f 3.2

```java
public class dessine extends Thread
{
    Graphics graphics;
    BufferStrategy strategy;
    Frame f1AR;
    String requete;

    public dessine(String requete,Frame frame) throws IOException
    {
        this.f1AR = frame;
        this.graphics = frame.getGraphics();
        this.strategy = frame.getBufferStrategy();
        this.requete = requete;
    }

    @Override
    public void run()
    {
        try
        {
            if("show".equals(this.requete.trim().toLowerCase()))
            {
                this.strategy.show();
            }
            else
            {
                String[] TabReq = this.requete.split(regex:",");

                if("window".equals(TabReq[0].trim().toLowerCase()))
                    this.Init(TabReq);
                else this.ReadResponse(TabReq);
            }
        }
        catch(Exception e){}
    }
    public void Init(String[] TabReq){ ···
    public boolean ReadResponse(String[] TabReq)
    {
        switch (TabReq[0].trim().toLowerCase())
        {
            case "droite": ···
            case "rectangle": ···
            case "croix": ···
            case "cercle": ···
            case "text": ···
            default: ···
        }
    }
```